
Unit, Widget and Integration Testing

Learning Outcomes

After completing this session you should be able to

- Define what Unit, Widget/Component, and Integration tests are
- Explain the tradeoffs of using Unit, Widget and Integration tests
- Write and run Unit Tests in Flutter
- Write and run Unit tests for Bloc
- Explain the advantages of Mocking
- Use Mockito package to mock dependencies

Learning Outcomes

After completing this session you should be able to

- Write and run widget tests
- Write and run integration tests

Testing Flutter apps

Automated testing falls into a few categories:

- A **unit test** tests a single function, method, or class
- A **widget/component test** tests a single widget
- An **integration test** tests a complete app or a large part of an app

Test Tradeoffs

	Unit	Widget	Integration
Confidence	Low	Higher	Highest
Maintenance cost	Low	Higher	Highest
Dependencies	Few	More	Most
Execution speed	Quick	Quick	Slow

Test Proportion

A well-tested app has **many unit and widget tests**, tracked by code coverage, plus **enough integration tests** to cover all the important use cases

Unit Test

The goal of a **unit test** is to **verify the correctness of a unit of logic under a variety of conditions**

External dependencies of the unit under test are generally **mocked out**

Unit tests generally don't read from or write to disk, render to screen, or receive user actions from outside the process running the test

For more information regarding unit tests, you can view the following recipes or run

```
flutter test --help
```

Widget Test

A **widget test** (in other UI frameworks referred to as component test) tests a single widget

The goal of a widget test is to **verify that the widget's UI looks and interacts as expected**

Testing a widget involves multiple classes and **requires a test environment that provides the appropriate widget lifecycle context**

Widget Test

The Widget being tested should be able to receive and respond to user actions and events, perform layout, and instantiate child widgets

A widget test is more comprehensive than a unit test

Like a unit test, **a widget test's environment is replaced with an implementation much simpler than a full-blown UI system**

Integration Test

An **integration test** tests a complete app or a large part of an app

The goal of an integration test is **to verify that all the widgets and services being tested work together as expected**

Furthermore, **you can use integration tests to verify your app's performance**

Integration Test

Generally, **an integration test runs on a real device or an OS emulator**, such as iOS Simulator or Android Emulator

The app under test is typically isolated from the test driver code to avoid skewing the results

General Testing Procedure in Flutter

To perform tests in flutter you can follow the following steps

- Add the required dependencies in the `pubspec.yaml`
- Create a test files (should have `_test.dart` suffix)
- Identify the classes, functions, widgets or flutter application that you want to test
- Write tests
- Combine multiple tests in a group if needed
- Run the tests

Unit Testing - Add the test dependency

The **test** package provides the core functionality for writing tests in Dart

Add the dependency shown below in your **pubspec.yaml** file or run the command:

```
flutter pub add test --dev
```

Unit Testing - Create a test file

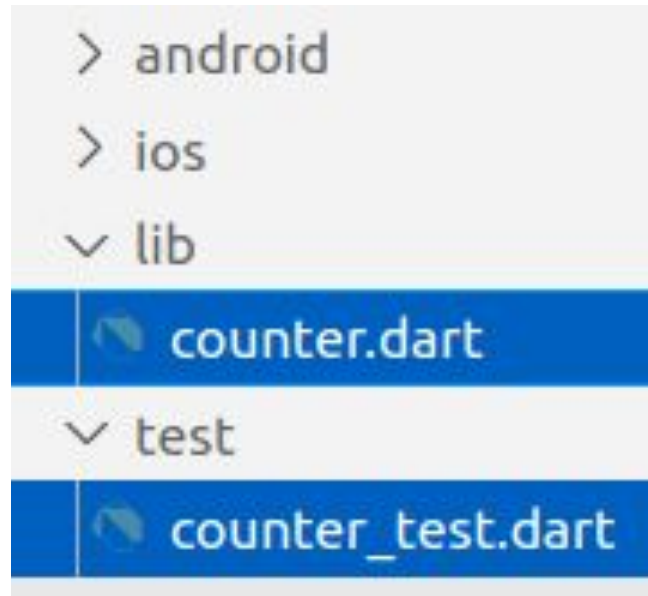
Let us assume we want to test the following class - a unit

```
class Counter {  
    int value = 0;  
    void increment() => value++;  
    void decrement() => value--;  
}
```

Test Directory and File Naming

In general, **unit test files** should reside inside a **test** folder located at the root of your Flutter application or package

Test files should always end with **_test.dart**, this is the convention used by the test runner when searching for tests



Writing Unit Tests

Tests are defined using the **top-level test function**, and you can check if the results are correct by using the **top-level expect function** that come from the **test** package

Writing Unit Tests

Inside the `counter_test.dart` file add the test code

```
import 'package:test/test.dart';
import 'package:counter_app/counter.dart';
void main() {
  test('Counter value should be incremented', () {
    final counter = Counter();
    counter.increment();
    expect(counter.value, 1);
  });
}
```

Grouping Multiple Tests

If you have several related tests you can combine them using the **group** function provided by the **test** package

```
void main() {  
  Run | Debug  
  group('Counter', () {  
    Run | Debug  
    test('Value should start at 0', () { ...  
    Run | Debug  
    test('Counter value should be incremented', () {  
      final counter = Counter();  
      counter.increment();  
      expect(counter.value, 1);  
    });  
    Run | Debug  
    test('Counter value should be decremented', () { ...  
  });  
}
```

Run tests using

You can run tests using the tools provided by IDEs or from terminal using `flutter test <file-path>` command as shown below

```
flutter test test/counter_test.dart
```

For more options regarding unit tests, you can execute this command:

```
flutter test --help
```

Unit Testing Bloc

Add `bloc_test` and `test` dependency

```
flutter pub add bloc_test --dev
```

```
flutter pub add test --dev
```

Bloc To Test

```
abstract class CounterEvent {}

class CounterIncrementPressed extends CounterEvent {}
class CounterDecrementPressed extends CounterEvent {}

class CounterBloc extends Bloc<CounterEvent, int> {
  CounterBloc() : super(0) {
    on<CounterIncrementPressed>(
      (event, emit) => emit(state + 1));
    on<CounterDecrementPressed>(
      (event, emit) => emit(state + 1));
  }
}
```

> android

> ios

✓ lib

 counter_bloc.dart

 main.dart

Bloc Test

We want to test the following three states of the bloc

- Nothing is emitted with no **CounterEvent**
- When **CounterIncrementPressed** event is added the bloc should emit **1**
- When **CounterDecrementPressed** event is added the bloc should emit **-1**

Bloc Test (counter_bloc_test.dart)

```
group('CounterBloc', () {  
  blocTest(  
    'emits [] when nothing is added',  
    build: () => CounterBloc(),  
    expect: () => [],  
  );  
  
  blocTest(  
    'emits [1] when CounterIncrementPressed is added',  
    build: () => CounterBloc(),  
    act: (bloc) => bloc.add(CounterIncrementPressed()),  
    expect: () => [1],  
  );  
});
```

Bloc Test (`counter_bloc_test.dart`)

```
blocTest<CounterBloc, int>(
  'emits [] when no event is added',
  build: () => CounterBloc(),
  expect: () => const [],
);
```


Bloc Test (counter_bloc_test.dart)

```
blocTest<CounterBloc, int>(
  'emit [1] when CounterIncrementPressed is added',
  build: () => CounterBloc(),
  act: (bloc) => bloc.add(CounterIncrementPressed()),
  expect: () => [1],
);
```

Bloc Test (`counter_bloc_test.dart`)

```
blocTest<CounterBloc, int>(
  'emit [-1] when CounterDecrementPressed() is added',
  build: () => CounterBloc(),
  act: (bloc) => bloc.add(CounterDecrementPressed()),
  expect: () => [-1],
);
```

https://pub.dev/packages/bloc_test/example

Mocking Dependencies

In most cases classes depend on other classes that fetch data from live web services or databases, for instance

This is inconvenient in case of unit testing for the following reasons

- Calling live services or databases **slows** down test execution (**Speed**)
- A passing test might start failing if a web service or database returns **unexpected results** (**Predictability/Reproducibility**)
- It is **difficult to test all possible success and failure scenarios** by using a live web service or database (**Control**)

Mocking Dependencies

For the cons mentioned, rather than relying on a live web service or database, it is better to “**mock**” these dependencies

Mocks allow emulating a live web service or database and return deterministic results depending on preconfigured setup/situation

You can mock dependencies by creating an alternative implementation of a class or you can use packages such as the **Mockito**

Mocking Example

Add the `mockito` and the `build_runner` package dependencies in your `pubspec.yaml` file or run the following commands

```
flutter pub add mockito --dev
```

```
flutter pub add build_runner --dev
```

The `build_runner` package is used for mock code generation

Function to test (`fetch_album.dart`)

```
import 'dart:convert';  
import 'package:http/http.dart' as http;  
import 'album.dart';  
  
Future<Album> fetchAlbum(http.Client client) async {  
    // ...  
}
```

You can get the `Album` class from
<https://flutter.dev/docs/cookbook/networking/fetch-data>

Function to test (`fetch_album.dart`)

```
Future<Album> fetchAlbum(http.Client client) async {  
    final response = await  
    client.get(Uri.parse('https://jsonplaceholder.typicode.com/albums/1'));  
  
    if (response.statusCode == 200) {  
        return Album.fromJson(jsonDecode(response.body));  
    } else {  
        throw Exception('Failed to load album');  
    }  
}
```

You can get the `Album` class from

<https://flutter.dev/docs/cookbook/networking/fetch-data>

Mocking `http.Client` (`fetch_album_test.dart`)

```
import 'package:http/http.dart' as http;  
import 'package:mockito/annotations.dart';
```

```
@GenerateMocks([http.Client])
```

```
void main() {  
}
```

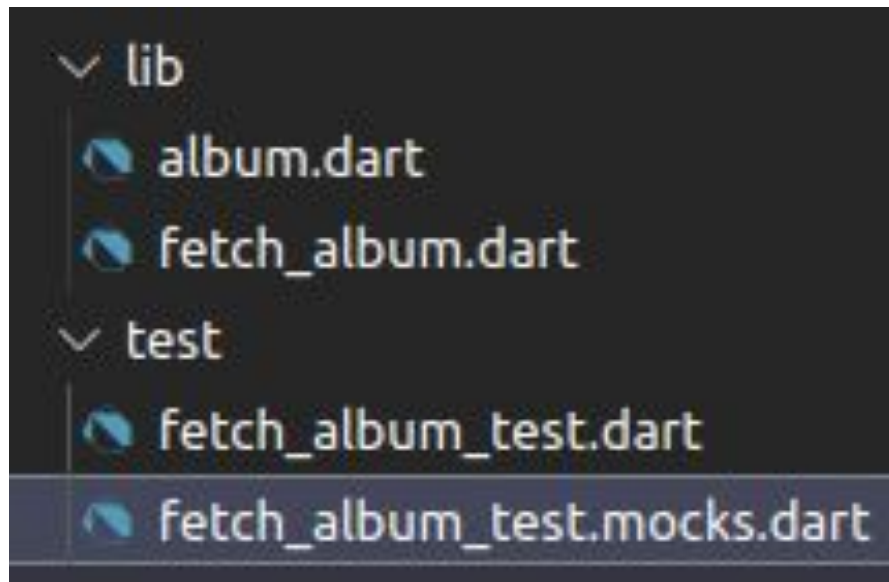
Run the following command to generate the mock file

```
flutter pub run build_runner build
```


Mocking `http.Client`

The last file shown below is the generated mock file

`fetch_album_test.mocks.dart`



Testing `fetchAlbum(http.Client)` function

The `fetchAlbum()` function does one of two things:

- Returns an `Album` if the http call succeeds
- Throws an `Exception` if the http call fails

We want to test these two conditions

Testing `fetchAlbum(http.Client)` function

Use the `MockClient` class to return an “ok” response for the **success** test, and an **error** response for the **unsuccessful** test

Test these conditions using the `when()` and `thenAnswer()` functions provided by `Mockito`

Testing `fetchAlbum(http.Client)` function

```
@GenerateMocks([http.Client])
void main() {
  Run | Debug
  group('fetchAlbum', () {
    Run | Debug
    test('returns Album if the http call completes successfully', () async { ...
    Run | Debug
    test('throws error if the http call completes with an error', () { ...
  });
}
```

Testing `fetchAlbum(http.Client)` function

```
test('returns an Album if the http call completes successfully', () async
{
    final client = MockClient();
    when(client
        .get(Uri.parse('https://jsonplaceholder.typicode.com/albums/1')))
        .thenAnswer((_) async =>
            http.Response('{ "userId": 1, "id": 2, "title": "mock" }', 200));
    expect(await fetchAlbum(client), isA<Album>());
});
```

Testing `fetchAlbum(http.Client)` function

```
test('throws an exception if the http call completes with an error',  
    () {  
        final client = MockClient();  
  
        when(client  
            .get(Uri.parse('https://jsonplaceholder.typicode.com/albums/1'))  
            .thenAnswer((_) async => http.Response('Not Found', 404)));  
  
        expect(fetchAlbum(client), throwsException);  
    });
```

Run the test using `flutter test test/fetch_album_test.dart`

Testing Widgets

The `flutter_test` package provides the following tools for testing widgets

- `WidgetTester`
- `testWidgets()`
- `Finder`
- `Matcher`

Testing Widgets

The `flutter_test` package provides the following tools for testing widgets

- The `WidgetTester` allows building and interacting with widgets in a test environment
- The `testWidgets()` function automatically creates a new `WidgetTester` for each test case, and is used in place of the normal `test()` function

Testing Widgets

The `flutter_test` package provides the following tools for testing widgets

- The `Finder` classes allow searching for widgets in the test environment
- Widget-specific `Matcher` constants help verify whether a `Finder` locates a widget or multiple widgets in the test environment

Add `flutter_test` Dependency

Add the `flutter_test` dependency in your `pubspec.yaml` file or run the following command

```
flutter pub add flutter_test --dev
```

```
dependencies:  
  flutter:  
    sdk: flutter  
  
dev_dependencies:  
  flutter_test:  
    sdk: flutter
```

A widget to test

```
class MyWidget extends StatelessWidget {  
  const MyWidget({  
    Key? key,  
    required this.title,  
    required this.message,  
  }) : super(key: key);  
  
  final String title;  
  final String message;  
  
  @override  
  Widget build(BuildContext context) { ...  
}
```

A widget to test

```
@override
Widget build(BuildContext context) {
  return MaterialApp(
    title: 'Flutter Demo',
    home: Scaffold(
      appBar: AppBar(
        title: Text(title),
      ),
      body: Center(
        child: Text(message),
      ),
    ),
  );
}
```

Create a testWidgets test

Put the following code in the test file (`my_widget_test.dart`)

```
void main() {  
    testWidgets('MyWidget has a title and message',  
    (WidgetTester tester) async {  
        // Test code goes here.  
    });  
}
```

Build the widget using the `WidgetTester`

```
void main() {  
    testWidgets('MyWidget has a title and message',  
        (WidgetTester tester) async {  
        // Create the widget by telling the tester to build it.  
        await tester.pumpWidget(  
            const MyWidget(title: 'Test Title', message: 'Test  
            Message' ));  
        });  
}
```

Build the widget using the **WidgetTester**

Variations of **tester.pumpWidget()** function

```
tester.pump(Duration duration)
```

```
tester.pumpAndSettle()
```

These methods provide fine-grained control over the build lifecycle

Search for widgets using a **Finder**

To verify that the widgets are being displayed correctly we need to find them

We are looking for **Text** widgets, therefore we should use the `find.text()` method

Search for widgets using a Finder

```
void main() {  
  testWidgets('MyWidget displays title and message',  
    (WidgetTester tester) async {  
      await tester  
        .pumpWidget(MyWidget(title: 'Test 123', message: 'Message 123'));  
  
      final tittleFinder = find.text('Test 123');  
      final messageFinder = find.text('Message 123');  
  
      expect(tittleFinder, findsOneWidget);  
      expect(messageFinder, findsWidgets);  
    });  
}
```

Other Finders

To find a widget with a specific **Key**

```
testWidgets('finds a widget using a Key', (WidgetTester tester) async {  
  // Define the test key.  
  const testKey = Key('K');  
  
  // Build a MaterialApp with the testKey.  
  await tester.pumpWidget(MaterialApp(key: testKey, home: Container()));  
  
  // Find the MaterialApp widget using the testKey.  
  expect(find.byKey(testKey), findsOneWidget);  
});
```

Other Finders

Find a specific **widget instance**

```
testWidgets('finds a specific instance', (WidgetTester tester)
  async {
    const childWidget = Padding(padding: EdgeInsets.zero);

    // Provide the childWidget to the Container.
    await tester.pumpWidget(Container(child: childWidget));

    // Search for the childWidget in the tree and verify it exists.
    expect(find.byWidget(childWidget), findsOneWidget);
  });
```

Verify the widget using a **Matcher**

```
void main() {  
  testWidgets('MyWidget has a title and message', (WidgetTester  
tester) async {  
    await tester.pumpWidget(const MyWidget(title: 'Test Title',  
message: 'Test Message'));  
    final titleFinder = find.text('Test Title');  
    final messageFinder = find.text('Test Message');  
  
    expect(titleFinder, findsOneWidget);  
    expect(messageFinder, findsOneWidget);  
  });  
}
```

Other Matchers

findsNothing

Verifies that no widgets are found

findsWidgets

Verifies that one or more widgets are found

Other Matchers

findsNWidgets

Verifies that a specific number of widgets are found

matchesGoldenFile

Verifies that a widget's rendering matches a particular bitmap image

Tap, drag, and enter text

Many widgets not only display information, but also respond to user interaction

This includes `buttons` that can be `tapped`, and `TextField` for `entering` text

To test these interactions, you can use the following methods from the `WidgetTester` class

- `enterText()`
- `tap()`
- `drag()`

Tap, drag, and enter text

```
testWidgets('Counter increments smoke test', (WidgetTester tester) async {  
  // Build our app and trigger a frame.  
  await tester.pumpWidget(MyApp());  
  
  // Verify that our counter starts at 0.  
  expect(find.text('0'), findsOneWidget);  
  expect(find.text('1'), findsNothing);  
  
  // Tap the '+' icon and trigger a frame.  
  await tester.tap(find.byIcon(Icons.add));  
  await tester.pump();  
  
  // Verify that our counter has incremented.  
  expect(find.text('0'), findsNothing);  
  expect(find.text('1'), findsOneWidget);  
});
```


Integration Testing

A **widget test** verifies the behavior of Flutter widgets **without running the app itself**

An **integration test** (also called **end-to-end** testing or **GUI** testing) **runs the full app**

Example Unit, Widget, and Integration Test

Check the following example to see all three tests together

<https://docs.flutter.dev/testing/>

References

<https://flutter.dev/docs/cookbook> (Testing section)

[Bloc State Management Library](#)

[bloc_test | Dart Package](#)

[How to test a Flutter app](#)

Reading Assignment

Testing Riverpod

<https://riverpod.dev/docs/cookbooks/testing>